



**Tecnológico
de Monterrey**

Evidencia1:

**Conceptos Básicos y Algoritmos
Fundamentales**

Allison Charlize Arriaza - A00844457

Profesor: David Alonso Cantú

20 de septiembre, 2026

Av. Eugenio Garza

Sada 2501 Sur, Tecnológico

64700 Monterrey, N.L.

**Programación de estructuras de datos y algoritmos
fundamentales**

(Grupo 607)

Casos de prueba

Prueba	Qué se probó	Resultado
1	log607-1.txt con Quick Sort	Los registros se ordenaron correctamente
2	log607-2.txt con Shell Sort	El archivo se ordenó sin errores
3	Búsqueda con un rango válido	Se generó range607.txt con los registros encontrados
4	Búsqueda con un rango sin datos	El programa mostró que no había registros
5	Realizar otra corrida	El programa volvió al menú sin cerrarse
6	Generación de output608.txt	Se creó el archivo con los registros ordenados

Evidencias

Figura 1.

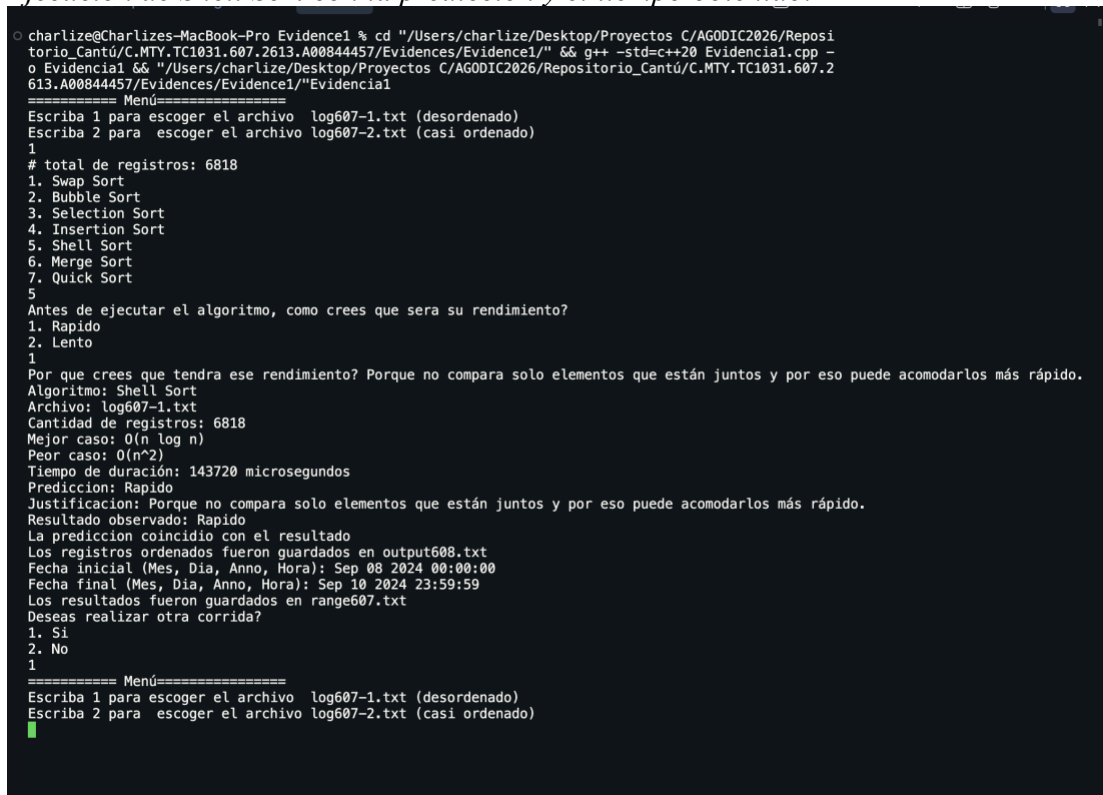
Menú para seleccionar el archivo y el algoritmo.



```
Problems Output Debug Console Terminal Ports Code - Evidence1 + - [] [X] ... | [C] [X]
charlize@Charlizes-MacBook-Pro Evidence1 % cd "/Users/charlize/Desktop/Proyectos C/AGODIC2026/Repositorio_Cantú/C.MTY.TC1031.607.2613.A00844457/Evidences/Evidence1/" && g++ -std=c++20 Evidencia1.cpp -o Evidencia1 && "/Users/charlize/Desktop/Proyectos C/AGODIC2026/Repositorio_Cantú/C.MTY.TC1031.607.2613.A00844457/Evidences/Evidence1/"Evidencia1
===== Menú=====
Escriba 1 para escoger el archivo log607-1.txt (desordenado)
Escriba 2 para escoger el archivo log607-2.txt (casi ordenado)
1
# total de registros: 6818
1. Swap Sort
2. Bubble Sort
3. Selection Sort
4. Insertion Sort
5. Shell Sort
6. Merge Sort
7. Quick Sort
```

Figura 2.

Ejecución de Shell Sort con la predicción y el tiempo obtenido.



```
charlize@Charlizes-MacBook-Pro Evidence1 % cd "/Users/charlize/Desktop/Proyectos C/AGODIC2026/Repositorio_Cantú/C.MTY.TC1031.607.2613.A00844457/Evidences/Evidence1/" && g++ -std=c++20 Evidencia1.cpp -o Evidencia1 && "/Users/charlize/Desktop/Proyectos C/AGODIC2026/Repositorio_Cantú/C.MTY.TC1031.607.2613.A00844457/Evidences/Evidence1/"Evidencia1
===== Menú=====
Escriba 1 para escoger el archivo log607-1.txt (desordenado)
Escriba 2 para escoger el archivo log607-2.txt (casi ordenado)
1
# total de registros: 6818
1. Swap Sort
2. Bubble Sort
3. Selection Sort
4. Insertion Sort
5. Shell Sort
6. Merge Sort
7. Quick Sort
5
Antes de ejecutar el algoritmo, como crees que sera su rendimiento?
1. Rapido
2. Lento
1
Por que crees que tendra ese rendimiento? Porque no compara solo elementos que están juntos y por eso puede acomodarlos más rápido.
Algoritmo: Shell Sort
Archivo: log607-1.txt
Cantidad de registros: 6818
Mejor caso: O(n log n)
Peor caso: O(n^2)
Tiempo de duración: 143720 microsegundos
Prediccion: Rapido
Justificación: Porque no compara solo elementos que están juntos y por eso puede acomodarlos más rápido.
Resultado observado: Rapido
La prediccion coincidio con el resultado
Los registros ordenados fueron guardados en output608.txt
Fecha inicial (Mes, Dia, Anno, Hora): Sep 08 2024 00:00:00
Fecha final (Mes, Dia, Anno, Hora): Sep 10 2024 23:59:59
Los resultados fueron guardados en range607.txt
Deseas realizar otra corrida?
1. Si
2. No
1
===== Menú=====
Escriba 1 para escoger el archivo log607-1.txt (desordenado)
Escriba 2 para escoger el archivo log607-2.txt (casi ordenado)
```

Figura 3.

Ejecución de Quick Sort mostrando mejor caso, peor caso y tiempo de ejecución.

```
Mejor caso: O(n log n)
charlize@Charlizes-MacBook-Pro Evidencia1 % cd "/Users/charlize/Desktop/Proyectos C/AGODIC2026/Repositorio_Cantú/C.MTY.TC1031.607.2613.A00844457/Evidences/Evidencia1/" && g++ -std=c++20 Evidencia1.cpp -o Evidencia1 && "/Users/charlize/Desktop/Proyectos C/AGODIC2026/Repositorio_Cantú/C.MTY.TC1031.607.2613.A00844457/Evidences/Evidencia1/"Evidencia1
La prediccion coincidio con el resultado
Los registros ordenados fueron guardados en output608.txt
Fecha inicial (Mes, Dia, Anno, Hora): Sep 08 2024 00:00:00
Fecha final (Mes, Dia, Anno, Hora): Sep 10 2024 23:59:59
Los resultados fueron guardados en range607.txt
Deseas realizar otra corrida?
1. Si
2. No
1
===== Menú=====
Escriba 1 para escoger el archivo log607-1.txt (desordenado)
Escriba 2 para escoger el archivo log607-2.txt (casi ordenado)
2
# total de registros: 6818
1. Swap Sort
2. Bubble Sort
3. Selection Sort
4. Insertion Sort
5. Shell Sort
6. Merge Sort
7. Quick Sort
7
Antes de ejecutar el algoritmo, como crees que sera su rendimiento?
1. Rapido
2. Lento
1
Por que crees que tendra ese rendimiento? Porque Quick Sort usa un pivote y con un archivo casi ordenado puede no dividir los datos tan bien.
Algoritmo: Quick Sort
Archivo: log607-2.txt
Cantidad de registros: 6818
Mejor caso: O(n log n)
Peor caso: O(n^2)
Tiempo de duracion: 2596042 microsegundos
Prediccion: Rapido
Justificacion: Porque Quick Sort usa un pivote y con un archivo casi ordenado puede no dividir los datos tan bien.
Resultado observado: Lento
La prediccion no coincidio con el resultado
Los registros ordenados fueron guardados en output608.txt
Fecha inicial (Mes, Dia, Anno, Hora): Sep 08 2024 00:00:00
Fecha final (Mes, Dia, Anno, Hora): Sep 10 2024 23:59:59
Los resultados fueron guardados en range607.txt
Deseas realizar otra corrida?
1. Si
2. No
2
charlize@Charlizes-MacBook-Pro Evidencia1 %
```

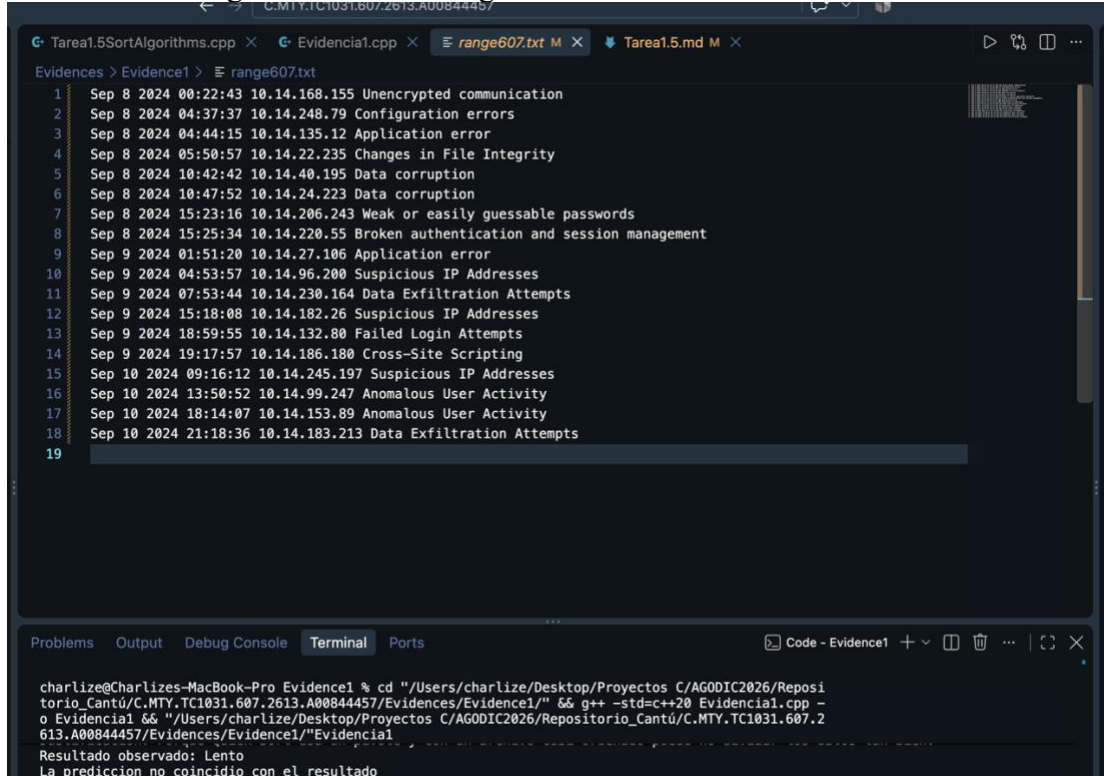
Figura 4.

Búsqueda por rango con resultados encontrados.

```
Los registros ordenados fueron guardados en output608.txt
Fecha inicial (Mes, Dia, Anno, Hora): Sep 08 2024 00:00:00
Fecha final (Mes, Dia, Anno, Hora): Sep 10 2024 23:59:59
```

Figura 5.

Contenido de range607.txt con los registros del intervalo seleccionado.

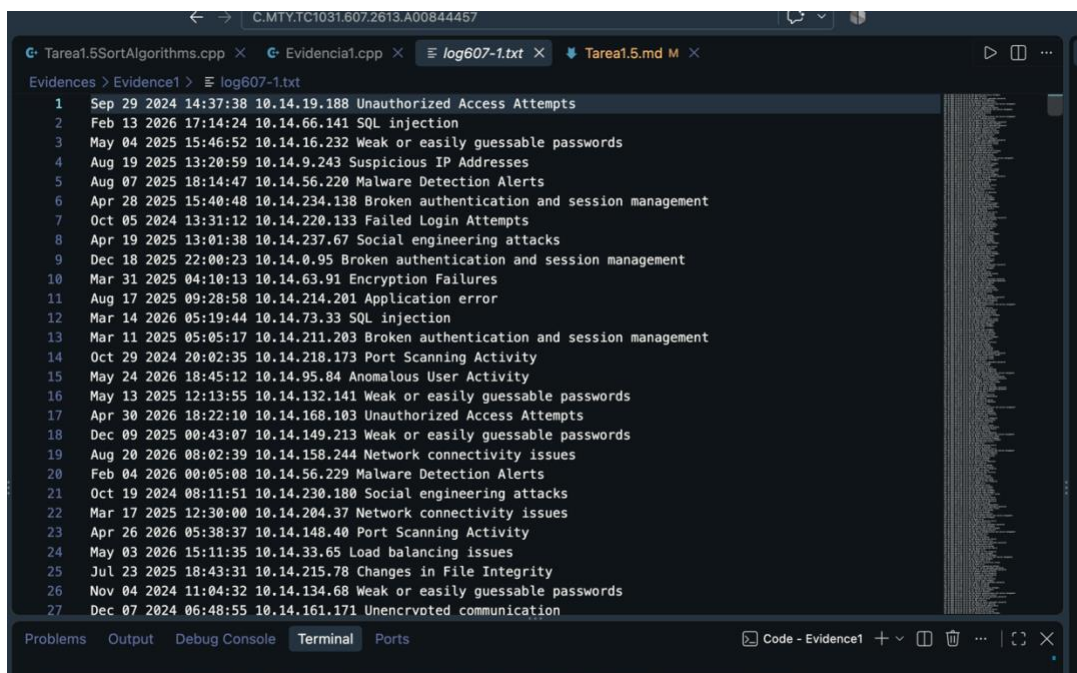


```
Evidences > Evidence1 > range607.txt
1 Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
2 Sep 8 2024 04:37:37 10.14.248.79 Configuration errors
3 Sep 8 2024 04:44:15 10.14.135.12 Application error
4 Sep 8 2024 05:50:57 10.14.22.235 Changes in File Integrity
5 Sep 8 2024 10:42:42 10.14.40.195 Data corruption
6 Sep 8 2024 10:47:52 10.14.24.223 Data corruption
7 Sep 8 2024 15:23:16 10.14.206.243 Weak or easily guessable passwords
8 Sep 8 2024 15:25:34 10.14.220.55 Broken authentication and session management
9 Sep 9 2024 01:51:20 10.14.27.106 Application error
10 Sep 9 2024 04:53:57 10.14.96.200 Suspicious IP Addresses
11 Sep 9 2024 07:53:44 10.14.230.164 Data Exfiltration Attempts
12 Sep 9 2024 15:18:08 10.14.182.26 Suspicious IP Addresses
13 Sep 9 2024 18:59:55 10.14.132.80 Failed Login Attempts
14 Sep 9 2024 19:17:57 10.14.186.180 Cross-Site Scripting
15 Sep 10 2024 09:16:12 10.14.245.197 Suspicious IP Addresses
16 Sep 10 2024 13:50:52 10.14.99.247 Anomalous User Activity
17 Sep 10 2024 18:14:07 10.14.153.89 Anomalous User Activity
18 Sep 10 2024 21:18:36 10.14.183.213 Data Exfiltration Attempts
19

charlize@Charlizes-MacBook-Pro: Evidence1 % cd "/Users/charlize/Desktop/Proyectos C/AGODIC2026/Repositorio_Cantú/C.MTY.TC1031.607.2613.A00844457/Evidences/Evidence1/" && g++ -std=c++20 Evidencial.cpp -o Evidencial && "/Users/charlize/Desktop/Proyectos C/AGODIC2026/Repositorio_Cantú/C.MTY.TC1031.607.2613.A00844457/Evidences/Evidence1/"Evidencial
Resultado observado: Lento
La prediccion no coincidio con el resultado
```

Figura 6.

Prueba de un rango sin registros y opción para realizar otra corrida.



```
Evidences > Evidence1 > log607-1.txt
1 Sep 29 2024 14:37:38 10.14.19.188 Unauthorized Access Attempts
2 Feb 13 2026 17:14:24 10.14.66.141 SQL injection
3 May 04 2025 15:46:52 10.14.16.232 Weak or easily guessable passwords
4 Aug 19 2025 13:20:59 10.14.9.243 Suspicious IP Addresses
5 Aug 07 2025 18:14:47 10.14.56.220 Malware Detection Alerts
6 Apr 28 2025 15:40:48 10.14.234.138 Broken authentication and session management
7 Oct 05 2024 13:31:12 10.14.220.133 Failed Login Attempts
8 Apr 19 2025 13:01:38 10.14.237.67 Social engineering attacks
9 Dec 18 2025 22:00:23 10.14.0.95 Broken authentication and session management
10 Mar 31 2025 04:10:13 10.14.63.91 Encryption Failures
11 Aug 17 2025 09:28:58 10.14.214.201 Application error
12 Mar 14 2026 05:19:44 10.14.73.33 SQL injection
13 Mar 11 2025 05:05:17 10.14.211.203 Broken authentication and session management
14 Oct 29 2024 20:02:35 10.14.218.173 Port Scanning Activity
15 May 24 2026 18:45:12 10.14.95.84 Anomalous User Activity
16 May 13 2025 12:13:55 10.14.132.141 Weak or easily guessable passwords
17 Apr 30 2026 18:22:10 10.14.168.103 Unauthorized Access Attempts
18 Dec 09 2025 00:43:07 10.14.149.213 Weak or easily guessable passwords
19 Aug 20 2026 08:02:39 10.14.158.244 Network connectivity issues
20 Feb 04 2026 00:05:08 10.14.56.229 Malware Detection Alerts
21 Oct 19 2024 08:11:51 10.14.230.180 Social engineering attacks
22 Mar 17 2025 12:30:00 10.14.204.37 Network connectivity issues
23 Apr 26 2026 05:38:37 10.14.148.40 Port Scanning Activity
24 May 03 2026 15:11:35 10.14.33.65 Load balancing issues
25 Jul 23 2025 18:43:31 10.14.215.78 Changes in File Integrity
26 Nov 04 2024 11:04:32 10.14.134.68 Weak or easily guessable passwords
27 Dec 07 2024 06:48:55 10.14.161.171 Unencrypted communication
```

Tabla de tiempos

Algoritmo	Log607-1.txt	Log607-2.txt
Swap Sort	21240244 microsegundos	7703107 microsegundos
Bubble Sort	21928079 microsegundos	34632461 microsegundos
Selection Sort	8013969 microsegundos	7656707 microsegundos
Insertion Sort	4138799 microsegundos	45182 microsegundos
Shell Sort	161124 microsegundos	92710 microsegundos
Merge Sort	117000 microsegundo	94257 microsegundos
Quick Sort	100973 microsegundos	2580467 microsegundos

Conclusión

En las pruebas se notaron diferencias notorias entre los tiempos de cada algoritmo. Por ejemplo en log607-1.txt, Quick Sort, Merge Sort y Shell Sort fueron de los más rápidos, mientras que Bubble Sort y Swap Sort tardaron más tiempo. Cuando se probó log607-2.txt que es un archivo que ya estaba casi ordenado, Insertion Sort mejoró bastante su tiempo de ordenamiento. Por otro lado, Quick Sort tardó más de lo esperado, lo cual puede estar relacionado con la forma en que se utiliza el pivote ya que en este método se utiliza el último elemento del arreglo. Cuando los datos ya están casi ordenados, ese pivote puede generar divisiones poco equilibradas, haciendo que el algoritmo tenga que realizar más comparaciones y tarde más tiempo. Esto permitió ver que el rendimiento de los algoritmos cambia dependiendo no solo del método que se utiliza, sino también del orden en el que vienen los datos.